EMS/VMS rev2c: Independent Pre-GPU Audit (Bengali)

বাংলা সারসংক্ষেপ: GPT-এর আগের অডিটের বাইরেও যা পাওয়া গেছে

চূড়ান্ত সিদ্ধান্ত: CPU-level fix-এ GO; পুরো GPU matrix থেকে winner নির্বাচন বা ১২-ঘণ্টার production engine অনুমোদনে এখনই NO-GO। এই সিদ্ধান্ত কোনো নতুন live-GPU crash-এর দাবি নয়; এটি audit harness-এর কয়েকটি গুরুত্বপূর্ণ detection ও gating সমস্যার ওপর ভিত্তি করে।

এই ZIP-এর `README_FOR_GPT.md`, EMS/VMS builders, patch, upstream source snapshots, local harness logs এবং CPU-kit report পরীক্ষা করেছি। নথিতে EMS **26/26** ও VMS **19/19** pass আছে। কিন্তু fake-server-এ বাস্তব GPU, FP8/GDN kernel বা recurrent speculative state execute হয়নি।

সবচেয়ে গুরুতর নতুন সমস্যা:

- **VMS আসল token ID না দেখে decoded string তুলছে।** vLLM 0.32.0-এর `return_token_ids=True` দিয়ে integer ID নিতে হবে।
- **Near-tie হলে দীর্ঘ ভুল continuation-ও `PASS` হতে পারে।** আমি মূল comparator-এর extracted function দিয়ে synthetic reproduction করেছি: প্রথম divergence-এর margin 0.2 nat, বাকিটা ভুল—তবুও verdict `PASS`। তাই margin-only gate production accuracy guarantee নয়।
- **Independent target-only oracle unavailable হলে EMS speculative DFlash-কে আবার reference ধরে।** Diagnostic comparison চলতে পারে, কিন্তু কোনো arm-কে adoptable বলা যাবে না।
- **PR #45151 patch apply ব্যর্থ হলেও `_pr45151` নামে arm চলবে।** Source marker/hash ও actual KV memory delta যাচাই ছাড়া এটিকে patched result বলা যাবে না।
- **VMS empty output detector-এ `if g.text` থাকায় empty stream বাদ পড়ে।** তিনটি empty output-এর synthetic test-এ পুরোনো count 0, সংশোধিত count 3।
- **8k থেকে 40k window-তে নোংরা server state লিক হতে পারে।** Stream shutdown-এর পরে thread join, idle count এবং prefill-settle verification প্রয়োজন।
- **Hybrid prefix caching-এ hit-rate 0% হওয়ার upstream evidence আছে।** শুধু `--enable-prefix-caching` দেখেই caching কাজ করছে ধরে নেওয়া যাবে না।

GPU quota খরচের আগে: E1–E8 ঠিক করুন; CPU negative regression যোগ করুন; তারপর ছোট bounded GPU smoke চালিয়ে `oracle_usable`, exact token-ID parity, KV-pool difference, empty/stream errors, actual running streams, `prefix_cache_hits_total` এবং দীর্ঘ context-এর একটি canary যাচাই করুন।

Claude-কে দেওয়ার নিয়ম: এই README-তে নিচে প্রতিটি source-এর সম্পূর্ণ `https://...` URL লেখা আছে; শুধু clickable title নয়। Code/flag/issue নাম মূল English বানানেই রাখা হয়েছে।

Date: 2026-10-26
Input: `ENGINE_MATRIX_EMS_VMS_REV2C_AUDIT_FOR_GPT_2026-10-26(1).zip`
Versions: SGLang 0.5.21 / vLLM 0.32.0; Qwen3.8-27B; four NVIDIA L4 GPUs, PCIe-only.
Scope: Beyond the supplied GPT audit: builder logic, gate semantics, source-level regressions, fake-server limitations, measurement reliability, run ordering.

Verdict: NO-GO for adopting an engine winner or interpreting the full matrix as correctness-certified. GO for CPU/static repairs. A bounded GPU smoke is appropriate **after** the HIGH repairs. No actual GPU run or notebook push was performed in this independent audit.

Evidence classification

- **Independently reproduced from included code:** The VMS near-tie corruption gate returns `PASS` on synthetic wrong continuations; the VMS steady-window degeneration count omits empty outputs. These are detector flaws, **not** proof the live model already returned wrong tokens.
- **Source-level confirmed:** Specific code paths in the supplied builders and source snapshots; remaining paths could occur under a described condition but are not known to have occurred on live GPUs.
- **Pack evidence, not independent GPU rerun:** The ZIP reports Kaggle CPU kit run 3: EMS 26/26 and VMS 19/19 (normal and bad-INT4 modes). The fake servers do not execute FP8/Marlin/GDN/MTP CUDA kernels. Source files are offline snapshots, not live wheel disassembly.
- **Upstream reports:** Linked GitHub sources describe observed bugs on various models/versions. Relevance to the pinned wheel requires testing; do not label every report an L4 reproduction.

Priority findings table

ID	Severity	Finding	File/location	Required action
E1	HIGH correctness	VMS compares decoded token strings rather than integer token IDs	`builders/build_vms.py:240-244, 407-435`	Request `return_token_ids`; read `choices[].token_ids`; reject missing IDs
E2	HIGH correctness	First-divergence margin can accept an arbitrarily corrupted continuation	`build_vms.py:407-435,560-575`; `build_ems.py:520-533`	Make near-tie outcomes `REVIEW_NUMERICS`, never `PASS` on margin alone
E3	HIGH reference	EMS falls back to speculative DFlash when target-only oracle is incomplete	`build_ems.py:564-604`	Fail closed: no `adoptable` arm without the independent target-only oracle
E4	HIGH configuration	#45151 patch failure does not prevent the “patched” arm running	`build_ems.py:282-323,613-616`	Skip patched arm until patch is applied, source verified, and GPU pool difference recorded

ID	Severity	Finding	File/location	Required action
E5	HIGH detector	Empty VMS streaming outputs are excluded from degenerate count	build_vms.py:350,379-394	Check every request, including empty streams; distinguish pending from abnormal
E6	HIGH measurement	VMS closes streams without joining and confirming a 0-running/0-waiting server	build_vms.py:269-352	Bounded joins and idle gate between 8k and 40k windows; abort contaminated windows
E7	HIGH measurement	VMS 120s settle loop starts a window even if prefill never settled	build_vms.py:308-321	Require a recorded settled=True; mark as incomplete on timeout
E8	HIGH decision	VMS a['ok'] means only two nonzero speed numbers, not correctness/ adoptability	build_vms.py:613-617; report 737-744	Separate measurement_ok, correctness_ok, adoptable
E9	MEDIUM/HIGH performance	Cache-hit assumption for shared 36k prefix is not measured; hybrid align mode can hit 0%	build_vms.py:651-653; vms_prom()	Record vllm:prefix_cache_hits_total, cached-token counters, cold/ warm hit rates per arm
E10	MEDIUM capacity	TP2 SGLang reports KV for 40k at reduced streams; does not prove 8×60k–90k production fit	build_ems.py:777-840	Label TP2 diagnostic; enforce actual per-replica context/ stream capacity
E11	MEDIUM coverage	No full 65k–155k or 60k+ generated-token corruption test; only 256/1024-token gates and 8k/40k bands	Both builders	Separate bounded long-context canary before 12h competition run
E12	MEDIUM budget	Time-priority order places risky FP8 NTN cold launch before DFlash20/24 and the best MTP retest	build_ems.py:613-629	Reorder by expected usefulness, confidence and cold-start cost

E1: VMS token-ID bug — minimal patch and test

The OpenAI-compatible `/v1/completions` `logprobs.tokens` field is `list[str]`. Strings are decoded surface forms; the current VGen does not collect integer token IDs. The vLLM completion protocol supports `return_token_ids` (including streaming deltas) and `return_tokens_as_token_ids`. The former is preferable.

```
# builders/build_vms.py — VGen.__init__
self.body['return_token_ids'] = True

# VGen.run, inside each choice
chunk_ids = c.get('token_ids')
if chunk_ids is None:
    self.err = 'MISSING_STREAM_TOKEN_IDS'
    break
if any(not isinstance(v, int) for v in chunk_ids):
    self.err = 'INVALID_STREAM_TOKEN_IDS'
    break
self.tokens.extend(chunk_ids)   # NOT: lp.get('tokens')
```

`logprobs.top_logprobs` remains available for margins. In the gate also assert `len(self.margins) >= len(self.tokens)` for the generated comparison prefix, unless the final chunk only contains metadata (do not invent missing margins). **Test:** Construct two chunks with distinct IDs but the same decoded string; the comparator must report a mismatch. Confirm both serial and concurrent requests, and ensure fake server emits real integer IDs.

Source: https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/entrypoints/openai/completion/protocol.py
https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/entrypoints/openai/completion/serving.py

E2: First-divergence margin is not a correctness proof — independently reproduced

Actual functions `vms_diverge`, `vms_degenerate`, and `vms_judge` were extracted from `builders/build_vms.py` without importing the GPU runtime. Synthetic oracle token strings `A B C D E`, candidate `A X X X X`, with both margins at the first difference 0.2 nat and nondegenerate text, produced:

```
FIRST-DIFF CORRUPTION VERDICT: PASS
compared: 6
large_margin_flips: 0
```

The implementation stops inspecting confidence at the **first** divergent token. A numerical near-tie may be harmless, but it can also lead to persistent corrupt state. Therefore margin-only PASS is unsound as a final accuracy guarantee.

```
def strict_adoption_gate(cmp_arm, cmp_aa):
    for r in (cmp_arm, cmp_aa):
        if not r or not r['expected'] or r['compared'] != r['expected'] \
           or r['errors'] or r['missing_margins']:
            return 'INCOMPLETE'
    if cmp_aa['identical'] != cmp_aa['expected']:
        return 'INCONCLUSIVE_ORACLE'  # even target-only A/A unstable
    if cmp_arm['identical'] == cmp_arm['expected']:
        return 'PASS_EXACT'
    return 'REVIEW_NUMERICS'          # cannot silently adopt on 1-nat threshold
```

Use the numeric-margin data for **triage**, then inspect full continuation, tool boundaries, and a small ARC task-success set before promoting `REVIEW_NUMERICS`. Different weight quantizations (FP8 vs INT4) require a separate task-accuracy control, not an exact-ID demand across checkpoints. **Tests:** (a) A/A exact, candidate near-tie then endless wrong tokens -> REVIEW; (b) all token IDs equal -> PASS; (c) missing margins -> INCOMPLETE; (d) noisy A/A -> INCONCLUSIVE.

Related upstream silent-output evidence:
https://github.com/vllm-project/vllm/issues/43559
https://github.com/vllm-project/vllm/issues/53912
https://github.com/sgl-project/sglang/issues/36548
https://github.com/sgl-project/sglang/issues/38009

E3: Fail closed when the independent target-only oracle is missing

EMS currently explicitly says that if the target-only A/A or probe is incomplete, it will keep the DFlash reference as `EMSR['oracle']='dflash_ref'`. That means target-only has not certified the speculative DFlash baseline, while subsequent DFlash/MTP arms can still pass against it. The fallback is useful for **diagnostic speed comparison**, but cannot authorize correctness/adoption.

```
# Immediately after the target-only oracle/probe selection, before PLAN
EMSR['oracle_usable'] = (
    EMSR.get('oracle') == 'target_only'
    and EMS_TGT is not None
    and _cmp_complete(EMSR.get('probe_tgt_aa'))
)

# At the START of _gated(a), before any speed-based selection
if not EMSR.get('oracle_usable'):
    return False
```

For VMS, a similar policy applies: if FP8 target-only oracle is incomplete or has degenerate responses, do not adopt any FP8 MTP arm based on the earlier potentially affected MTP+prefix reference. **Test:** Force the target-only oracle to return one missing row; every arm must have `adoptable=False`; diagnostic speed measurements may continue.

E4: PR #45151 backport verification

The local source confirms SGLang 0.5.21 scheduler calls `init_target_memory_pool()` before `EagleDraftWorker.alloc_memory_pool()`. The supplied rev2c patch shifts `init_token_map()` and `init_lm_head()` into `EagleDraftWorker.__init__()` under `EMS_PR45151=1`, then skips the late duplicate call. This resembles the upstream PR's two hunks and **is plausible**. It is NOT verified until actual GPU memory profiling shows the freed allocations before KV sizing.

New defect: `ems_patch_pr45151()` can return `{'applied': False, 'why': ...}`, but the fixed `PLAN` still launches `mtp_3_1_4_pr45151` with `EMS_PR45151=1`. A failed site-packages rewrite thus becomes a falsely labeled patched benchmark.

```
# In the PLAN loop, before invoking _arm
if _name == 'mtp_3_1_4_pr45151':
    patch = EMSR.get('pr45151_patch') or {}
    if not patch.get('applied'):
        EMSR['arms'][_name] = {
            'ok': False, 'gate': 'INCOMPLETE',
```

```
            'skipped': 'PR45151_NOT_APPLIED: ' + str(patch.get('why')),
            'adoptable': False,
        }
        EMSR['order'].append(_name)
        continue
```

Further hardening: when the marker string `EMS_PR45151` already appears, verify **both** modified code anchors and the expected source hash, not just the presence of that substring. Stage changes to a temporary file and use an atomic `os.replace` if possible. On a real GPU startup record the worker's `EMS_PR45151` status, `torch.cuda.mem_get_info()` before/after share, actual available KV tokens before graph capture, and output identity. A `patch applied` flag alone is insufficient.

Source: https://github.com/sgl-project/sglang/pull/45151
https://github.com/sgl-project/sglang/issues/44145

E5: Empty-output detector bypass — independently reproduced

The steady-window code currently calls:

```
vms_degenerate([''.join(g.text) for g in gens if g.text])
```

Because of `if g.text`, clients producing zero text are excluded **before** `vms_degenerate()` can count empties. Synthetic test with three empty generators:

```
Current filter: degenerate=0
Correct all-request input: degenerate=3
```

```
# Include every request, then classify errors separately
texts = [''.join(g.text) for g in gens]
out['empty_outputs'] = sum(not t.strip() for t in texts)
out['degenerate'] = vms_degenerate(texts)
out['stream_errors'] = sum(bool(g.err) for g in gens)
```

Important: A 75-second speed-window may intentionally abort some incomplete requests. Count `no_first_token`, `incomplete`, `errored`, and `finished_empty` separately. To assert the #53912 silent-empty bug, use a non-aborted validation request with normal `finish_reason` and empty content. Do not automatically equate an unstarted/aborted request with model corruption.

Source: https://github.com/vllm-project/vllm/issues/53912

E6-E7: Window contamination and prefill settle

`vms_window()` launches live stream threads, stops them in a `finally` block, but never joins them and waits only 3 seconds before returning. There is no assertion that the vLLM scheduler has zero `num_requests_running` and zero `num_requests_waiting`. A previous 8k window can therefore leave residual work or delayed metrics updates in the following 40k window. Also `t_s < 120` is a timeout that drops through to measurement even if the prompt-token metric is still changing.

```
# After each window: bounded teardown and a clean-idle gate
for g in gens:
    g.stop()
for g in gens:
```

```
    g.join(timeout=5)
if any(g.is_alive() for g in gens):
    out['error'] = 'STREAM_THREADS_NOT_JOINED'
    return out

deadline = time.perf_counter() + 30
idle_samples = 0
while time.perf_counter() < deadline:
    p = vms_prom()
    idle = (_met(p, 'vllm:num_requests_running') == 0 and
            _met(p, 'vllm:num_requests_waiting') == 0)
    idle_samples = idle_samples + 1 if idle else 0
    if idle_samples >= 3:
        break
    time.sleep(2)
if idle_samples < 3:
    out['error'] = 'DIRTY_SERVER_AFTER_WINDOW'
    return out
```

Implement this outside an unconditional `return` path; in particular, **always run the teardown in `finally`** while preserving the error. For prefill, add an explicit `settled` flag and require three consecutive stable `prompt_tokens_total` observations with the desired running/queue state; a 120s timeout should be `PREFILL_NOT_SETTLED`, not a normal benchmark. When queue plateau is intentionally allowed, label throughput as **capacity-limited**, never the requested level.

Tests: Fake server with delayed cancellation (e.g., remains running 12s after socket shutdown); test must block or mark dirty, never produce a clean long-band result. Fake 121s continuous prefill must flag the window incomplete.

E8: Separate measurement success from adoption

`vms_arm` currently sets `a['ok'] = all(tok_s for short,long)` even if `gate='FAIL'`, `stream_errors>0`, `degenerate>0`, or it did not reach requested concurrency. That design is valid only if `ok` explicitly means "speed data exists". Human readers and downstream consumers may otherwise select a faster but incorrect arm.

```
a['measurement_ok'] = bool(
    all((a.get(b) or {}).get('tok_s', 0) > 0 for b in ('short','long'))
    and not any((a.get(b) or {}).get('error') for b in ('short','long'))
)
a['correctness_ok'] = (a.get('gate') == 'PASS')
a['capacity_ok'] = all(
    (a.get(b) or {}).get('measured_level') == a['level_wanted']
    for b in ('short','long')
)
a['adoptable'] = bool(
    a['measurement_ok'] and a['correctness_ok'] and a['capacity_ok']
    and a.get('server_alive_after') and not a.get('log_errors')
    and all((a.get(b) or {}).get('stream_errors', 0) == 0 for b in ('short','long'))
)
```

Caution: For DP2×TP2 you must evaluate **per-DP-rank** capacity and errors, not only aggregate totals, and the reference/oracle rules differ for INT4 vs FP8. `gate='diagnostic'` is never correctness certification. Do not choose the largest `tok_s` without filtering on `adoptable`.

E9: Prefix caching might be a silent no-op

The long-band shared 36k prefix is useful, but the VMS matrix does not record cache-hit counters. Hybrid align-mode prefix caching depends on GDN/Mamba checkpoint alignment, and real upstream reports show some MTP+prefix configurations with 0 cache hits despite shared prefixes. Cache hits cannot be inferred from the flag or from a repeated prompt alone.

```
# Record deltas between p0 and p1 in the speed window
out['prefix_hit_tokens'] = (
    _met(p1, 'vllm:prefix_cache_hits_total') -
    _met(p0, 'vllm:prefix_cache_hits_total')
)
out['cached_prompt_tokens'] = (
    _met(p1, 'vllm:prompt_tokens_cached_total') -
    _met(p0, 'vllm:prompt_tokens_cached_total')
)
```

Also collect cold/warm **time-to-first-token**, measured prompt-token throughput, identical-prompt hit counts, and per-engine cache counters for DP. `prefix_hit_tokens == 0` does not itself imply wrong tokens; it can be a severe silent speed regression.

Sources:
https://github.com/vllm-project/vllm/issues/45238
https://github.com/vllm-project/vllm/issues/51250
https://github.com/vllm-project/vllm/issues/54360

E10-E12: Capacity, realistic ARC validation, and time budget

2×TP2: The matrix's long band deliberately reduces per-replica streams based on `min(KV tokens)//45000` (`build_ems.py:778-780`). A run reported at **reduced long concurrency** cannot establish that each TP2 server will hold eight 60k-90k reasoning streams. Keep `kv_ok_for_production`; reject capacity claims unless actual per-replica longest-case workload fits. The rough TP2 ~115k-170k KV-token estimates from the pack are hypotheses, not measured results.

Long-context coverage: 256-token gate, 1024-token probe, 8k and 40k windows can miss >64k-context and later-stage 60k-90k generated-token errors. Before a 12-hour ARC run, reserve a small direct request at ~65k and ~100k context, plus a longer churn/resume check with real ARC tool-call format; inspect actual output IDs and request health. Do not run the entire 90k sequence in every matrix arm.

Production realism: 75s steady windows and `ignore_eos=True` stream generators measure serving capacity, not task-success tokens or task completion. Measure at least one native ARC/Poetiq workload at temperature 1, top_p=.95, top_k=20, with tool interleaving for finalists. The greedy gate is for correctness isolation, **not** an exact reproduction of stochastic production.

Ordering to protect 200 min:

Rank	EMS (SGLang)	VMS (vLLM)
1	Target-only oracle and DFlash W6 reference integrity	FP8 target-only oracle and MTP3 baseline integrity
2	MTP 3/1/4 vs patched #45151 (only if patch verified)	FP8 MTP3 no-prefix (isolates known cache risk)

Rank	EMS (SGLang)	VMS (vLLM)
3	DFlash W6 at 20 streams	FP8 MTP2, then MTP4
4	DFlash W6 at 24 if 20 passes	FP8 MTP3 at 20 streams
5	Best gated MTP at 20	FP8 MTP3 at 24 if 20 passes
6	TP2 DFlash diagnostic	FP8 Triton-vs-Marlin kernel comparison
7	NTN FP8/INT4, ReplaySSM, width4/8	INT4+MTP (static check only screens loader risk), TP2 diagnostic

Keep existing TP2 reserve only if TP2 has priority; otherwise explicitly schedule it as the final diagnostic and never call the previous skipped arms a full sweep.

Notes on the #45151 patch versus upstream

The upstream PR is currently reported **open**. It shares draft embed/head once before the target memory pool is sized and reuses the shared state during draft allocation. The pack's implementation is env-gated rather than the PR's unconditional single-state helper, and calls `torch.cuda.empty_cache()` after sharing. This is reasonable as a **hypothesis**, but must be validated with kernel startup and real KV token counts. The CPU kit explicitly skips the installed SGLang patch because its fake test environment has no SGLang package. Therefore the 26/26 result is **not** evidence the on-GPU patch applied or released memory.

- Upstream source: https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/speculative/eagle_worker_v2.py
- Scheduler: https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/managers/scheduler.py
- Patch: https://github.com/sgl-project/sglang/pull/45151
- Underlying issue: https://github.com/sgl-project/sglang/issues/44145

Correctly fixed / not new blockers

- RedHat static ignore list `re:^mtp.*` covers the 15 listed dense `mtp.*` keys. This is an appropriate static exclusion check for the supplied checkpoint; **not** proof the CUDA MTP loader will succeed.
- The CPU-kit caught the Python 3.12 `Thread._stop` shadowing bug; the current `VGen._halt` avoids that bug.
- The vLLM 0.32.0 source supports `--no-enable-prefix-caching` and `--linear-backend triton`; removed `VLLM_TEST_FORCE_FP8_MARLIN` arm was rightly dropped.
- TP2 custom all-reduce disabled is a reasonable conservative baseline given the pack's measured P2P/IPC penalty; speedup from two TP2 replicas remains unproven.
- The grammar-boundary-specific #43029 is not by itself a blocker for an ordinary non-grammar workload. Other speculative GDN-state risks remain.

Final decision

No hard production engine recommendation from these fake-server results. Fix E1-E8 first; validate the patch's actual application and the independent oracle; add prefix-cache metrics and a long-context canary. Then run **a bounded GPU preflight**, not the full 200-min sweep. For every arm preserve: `oracle_usable`, `gate_exact_match`, `review_numerics`, `measurement_ok`, `adoptable`, `actual_max_running`, `peak_gpu_memory`, `KV tokens`, `prefix cache hits`, `finished_empty`, `stream_errors`, and the exact launch flags and hashes.

Source URLs (all visible, copyable)

1. https://github.com/sgl-project/sglang/pull/45151
2. https://github.com/sgl-project/sglang/issues/44145
3. https://github.com/sgl-project/sglang/issues/36548
4. https://github.com/sgl-project/sglang/issues/38009
5. https://github.com/vllm-project/vllm/issues/58807
6. https://github.com/vllm-project/vllm/issues/43559
7. https://github.com/vllm-project/vllm/issues/53912
8. https://github.com/vllm-project/vllm/issues/45238
9. https://github.com/vllm-project/vllm/issues/51250
10. https://github.com/vllm-project/vllm/issues/54360
11. https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/entrypoints/openai/completion/protocol.py
12. https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/entrypoints/openai/completion/serving.py
13. https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/speculative/eagle_worker_v2.py
14. https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/managers/scheduler.py

Reproduction performed: lightweight CPU synthetic calls to the actual extracted VMS comparator/heuristic only. **Not performed:** original 19/26 fake-server rerun, real-GPU start, direct Kaggle task inference, any GitHub push.